

AWS, Azure ML, Kubernetes)

A typical ML project looks like this:

```
project/
├─ data/
├─ notebooks/
├─ models/
├─ mlruns/ ← MLflow stores your experiment logs here
└─ train.py
```

MLflow automatically records each experiment run; no manual book-keeping is needed.

Tracking experiments – the heart of MLflow

Experiment tracking is where MLflow shines.

When you train a model, MLflow logs:

- Parameters (learning rate, depth, seed, etc)
- Metrics (accuracy, RMSE, ROC-AUC)
- Artifacts (plots, model files, configs)

- Environment (Python version, library versions)
- Code snapshot (if enabled)
This helps teams answer questions like:
 - Why did Experiment #7 outperform Experiment #4?
 - What version of the dataset was used?
 - Which model is safe to promote to staging?

Instead of writing everything to a spreadsheet or guessing from memory, MLflow stores it and visualises it automatically.

Visualising and comparing runs

This is the feature that makes MLflow indispensable.

Teams can:

- Sort runs by accuracy or loss
 - Compare hyperparameter combinations
 - See learning curves
 - Dive deep into each run's metadata
- A sample MLflow experiment run is shown in Figure 2.

Model Registry: The control centre for versioning

MLflow's Model Registry turns experiment results into production assets.

You can:

- Register a model
- Assign stages: Staging, Production, Archived
- Add comments and approval notes
- Track which version is currently in production

This prevents the classic mistake:

Shipping the wrong model to production. It also allows multiple teams to collaborate without stepping on each other.

CI/CD integration: Automating the ML lifecycle

MLflow fits naturally into DevOps pipelines. Using GitHub Actions, Azure Pipelines, Jenkins, or GitLab CI, you can automate:

- Training
- Logging
- Registering new models

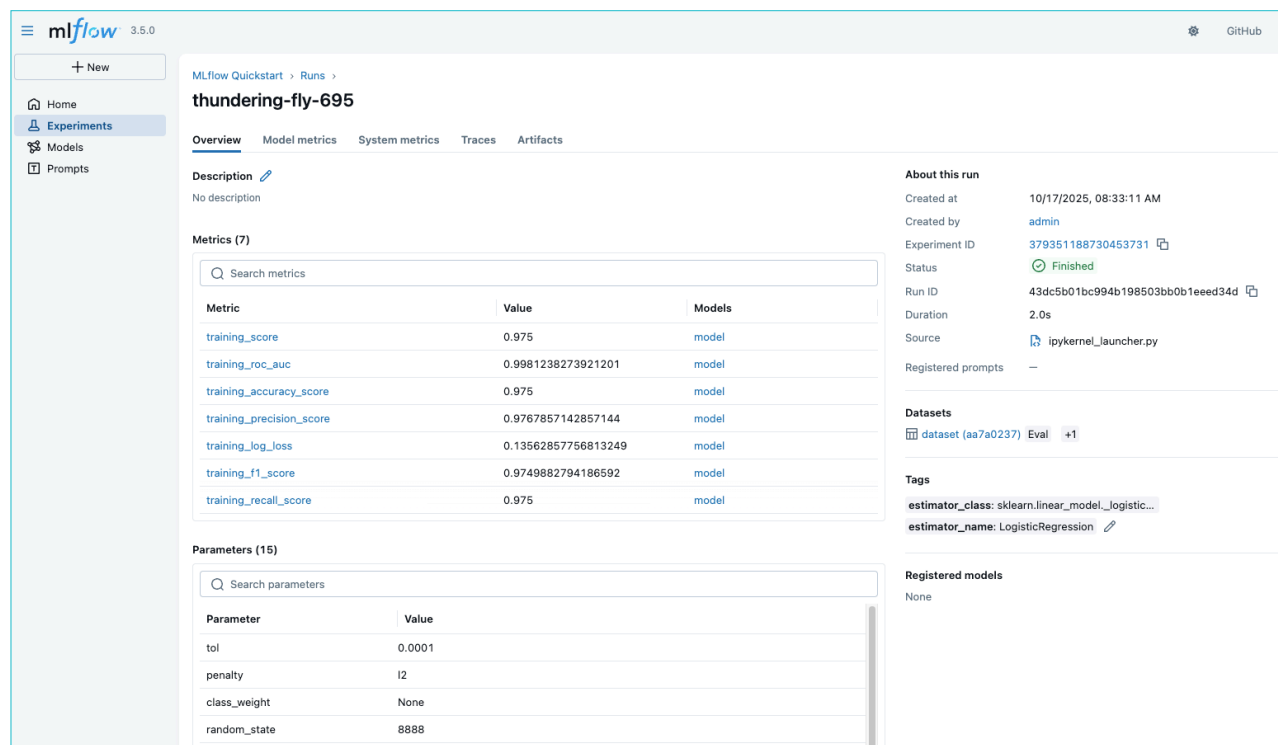


Figure 2: MLflow run page showing logged metrics, parameters, and artifacts (Source: <https://mlflow.org/docs/latest/ml/tracking/quickstart/>)